Ayudantía 2

Clases y Objetos

Programación Avanzada | Sección: 03 | CIT 1010
Universidad Diego Portales

Contacto: isaias.vasquez1@mail.udp.cl Discord: _iszzza

Github: github.com/Isza285/Programacion-Avanzada-2026-01

Sección 1

Clases

Clases

¿Qué es una clase?

Una clase se puede tratar como un molde, el cual nosotros le daremos atributos y métodos(una función dentro de la clase).

```
class Auto{ //Declaración de la clase
public: //Acceso público
  int año;//Atributo
  int potencia;//Atributo
  void mostrarInfo( ){//Método
    cout<<"Año: "<<año<<endl;
    cout<<"Potencia:"<<potencia;
  }
};
```

Modificadores de acceso

Los modificadores de acceso definen desde donde se puede acceder a los atributos y métodos de una clase.

- Public: Acceso libre
- Private: Solo desde la misma clase
- Protected: Desde la misma clase y clases hijas

```
class Persona{
public:
    string nombre;

private:
    int edad;

protected:
    string direccion;
};
```

```cpp
class Auto{
public:
    int año;
private:
    int potencia;
};

int main(){

    Auto toyota;
    toyota.año = 2020;
    toyota.potencia = 200;

    return 0;
}
```

```
main.cpp:17:12: error: ‘int Auto::potencia’ is private within this context
   17 |     toyota.potencia = 200;
      |            ^~~~~~~~
main.cpp:9:9: note: declared private here
    9 |     int potencia;
      |         ^~~~~~~~
```

Al intentar modificar el atributo *potencia* ocurre un error, debido a que la variable es privada.

<u>Nota importante:</u>

Si al momento de crear una clase no se le asigna un modificador de acceso(Ej: public) a ninguno de los atributos o métodos, estos por defecto se declararan como *private*

Sección 2

Objetos

Objetos

Un objeto es una instancia creada a partir de una clase previamente definida

Para acceder a los atributos se utiliza un punto (.)

ej: nombreObjeto.atributo

```
toyota.año = 2020;
toyota.potencia = 200;
```

```
class Auto{
public:
    int año;
    int potencia;
};

int main(){
    Auto toyota;//Creación del
objeto toyota que es de tipo Auto
}
```

Constructor

Al momento se hacer una instancia de un objeto, si no se define un constructor, los atributos de este objeto no tendrán valores y se deberá modificar directamente.

Al realizar un constructor, cuando creamos el objeto podremos asignarle sus atributos de manera inmediata

```
class Auto{
public:
    int año;
    int potencia;

    Auto(int a, int p){
        año = a;
        potencia = p;
    }
};

int main(){

    Auto toyota(2020,200);

    return 0;
}
```

Sección 3

Objetos compuestos

Objetos compuestos

Un objeto compuesto es un objeto que contiene otros objetos dentro de sus atributos

```
class Motor{
public:
    int potencia;
};

class Auto{
public:
    Motor motor;
    int año;
};
```

Se puede realizar un acceso encadenado utilizando puntos (.):

```
toyota.Motor.potencia = 200;
```

Sección 4

Ejemplo (Control 1 2024-02)

Una nueva empresa de delivery, le pide a usted diseñar su programa para la gestión de los envíos. Particularmente, se le pide crea la clase **Dirección** que tenga los siguientes atributos:

- calle
- número
- ciudad
- codigoPostal

Además, debe incluir los siguientes métodos públicos:

- Un constructor que inicialice todos los atributos.
- get para cada atributo

Por otra parte, se debe tener una clase **Persona** que tenga los siguientes atributos:

- nombre
- edad
- Un objeto de la clase Dirección como atributo.

la clase Persona debe tener los siguientes métodos:

- Un constructor que inicialice el nombre, la edad y la dirección.
- Un método mostrarInformacion que imprima los detalles de la persona y su dirección.

```
class Direccion{ //Definiendo la clase
public:
    //Atributos
    string calle;
    int numero;
    string ciudad;
    int codigoPostal;
    //Constructor
    Direccion(string c, int n, string ci, int co){
        calle = c;
        numero = n;
        ciudad = ci;
        codigoPostal = co;
    }
    //Metodos
    string getCalle(){return calle;}
    int getNumero(){return numero;}
    string getCiudad(){return ciudad;}
    int getCodigoPostal(){return codigoPostal;}
};
```

```
class Persona{
public:
    string nombre;
    int edad;
    Direccion direccion;
    //Constructor utilizando lista de inicialización
    Persona(string n, int e, Direccion d): nombre(n), edad(e), direccion(d){}

    void mostrarInformacion(){
        cout<<"Nombre: "<<nombre<<endl;
        cout<<"Edad: "<<edad<<endl;
        cout<<"Calle: "<<direccion.getCalle()<<endl;
        cout<<"Numero: "<<direccion.getNumero()<<endl;
        cout<<"Ciudad: "<<direccion.getCiudad()<<endl;
        cout<<"Codigo postal: "<<direccion.getCodigoPostal()<<endl;
    }

};
```

Distintos constructores

Se pueden hacer de dos formas:

- Asignación en el cuerpo (Llaves {}): Primero crea el objeto con valores basura y luego los sobrescribe
- Lista de inicialización (:): construye el objeto directamente en la memoria con sus valores predefinidos

En el ejemplo si no se utiliza una lista de inicialización en la clase de persona ocurrirá un error

```
main.cpp: In constructor ‘Persona::Persona(std::string, int, Direccion)’:
main.cpp:32:42: error: no matching function for call to ‘Direccion::Direccion()’
   32 |     Persona(string n, int e, Direccion d){
      |                                          ^
```

```
class Direccion{
public:
    string calle;
    int numero;
    string ciudad;
    int codigoPostal;
    //Constructor en el cuerpo
    Direccion(string c, int n, string ci, int co){
        calle = c;
        numero = n;
        ciudad = ci;
        codigoPostal = co;
    }
    //Constructor con lista de inicialización
    Direccion(string c, int n, string ci, int co): calle(c), numero(n), ciudad(ci), codigoPostal(co){}
};
```

Sección 5

Ejercitación

Ejercicio 1

Crear la Clase Estudiante que contenga:

- Nombre
- Edad
- Promedio (float)

Crear un constructor que inicialice todos sus atributos

En el main crear un objeto utilizando el constructor y mostrar los datos del estudiante.

Ejercicio 2

Crea una clase de una mascota (ejemplo: class Gato, class Perro), el cual debe tener los siguientes atributos:

- edad
- nombre
- nombreDueño

Ademas crear constructor que inicialice todos sus atributos y un constructor por defecto, métodos get, set y un método mostrarInformacion.

Por ultimo instanciar un objeto de la clase recién creada, asignarle valores a sus atributos y utilizar el método mostrarInformacion.

Ejercicio 3

Crear una clase Motor que tenga:

- potencia

Crear una clase Auto que tenga:

- marca
- Año
- objeto Motor

Crear un constructor que inicialice todos sus atributos, métodos get y set para Motor y Auto, y un método mostrarInformacion.

En el main crear una instancia del objeto Auto y mostrar su información.